

Universidad Nacional Autónoma de México

FACULTAD DE CIENCIAS, 2027-I

COMPILADORES



PRÁCTICA 02

ANALIZADOR LÉXICO COMPLETO DE MINIC

EQUIPO:

Delgado Díaz Hermes Alberto - 319258613

Medina Peralta Joaquín - 320202513

11 de septiembre de 2026

1. Objetivo

El objetivo de la práctica fue construir el analizador léxico de MiniC, manteniendo la implementación de la práctica pasada y agregando nuevas funcionalidades como la identificación de identificadores de variables, palabras reservadas y operadores compuestos, también de omitir los comentarios de una sola línea.

Se extendió la funcionalidad de `lexer.c` para dejar más modular para el escaneo de caracteres del programa por cada uno de los tipos de tokens.

2. Evolución respecto de la Práctica 1

Principalmente se modificó el módulo de `lexer.c`, en el cuál se modifica la función principal de `lexer_scan(FILE *file)` para agregar los nuevos casos y su manejo, de este modo, se dejó lo más simple posible para su posterior mantenimiento dejando las funciones de estos casos por separado de la función principal de nuestro `lexer`.

No se realizó el cambio de ninguna representación de tokens, debido a que desde la anterior práctica ya estaba definido todos los casos que maneja el lenguaje de MiniC. Además, que no se realizó ninguna re factorización importante ni correcciones.

3. Descripción de la solución

3.1. Reconocimiento de identificadores

Lo que se hizo para resolver el tema de los identificadores, primero tenemos que identificar si el carácter es candidato a ser el inicio de un identificador el cual, como indica la expresión regular del lenguaje, tiene que ser un valor del alfabeto o un `_`.

En el caso de no ser el inicio de un identificador, continua con el flujo, en otro caso, se tiene que inicializar un valor de lexema con 16 espacios en memoria, posteriormente, similar a la identificación de un número, se tiene que leer un carácter del archivo, teniendo los siguientes casos:

- Si es un final de archivo o no es parte de un identificador (tiene que ser un valor alfanumérico o `_`), se debe de regresar el carácter al buffer de lectura del archivo solo si no es un carácter final.
- En otro caso, se tiene que verificar si el tamaño del lexema nos permite guardar el carácter leído, de no ser posible, se tiene que reasignar su capacidad en memoria. Después de ese paso, se agrega el carácter al lexema y se continua leyendo el buffer del archivo.

3.2. Distinción entre identificadores, palabras reservadas y booleanos

Para esto, al momento de leer un lexema del tipo identificador, se debe de clasificar para regresar un token del tipo palabra reservada o un token de identificador, lo cual se hace antes de inicializar nuestra estructura de token.

De este modo, se decidió primero contar la cantidad de caracteres del lexema registrado y de ahí se tiene los siguientes casos:

```
switch (len) {
    case 2:
        if (lexeme[0] == 'i' && lexeme[1] == 'f') return IF;
        break;
    case 3:
        if (lexeme[0] == 'i' && lexeme[1] == 'n'
            && lexeme[2] == 't') return INT;
        break;
    case 4:
        if (lexeme[0] == 'b' && lexeme[1] == 'o'
            && lexeme[2] == 'o' && lexeme[3] == 'l') return BOOL;
        if (lexeme[0] == 'e' && lexeme[1] == 'l'
            && lexeme[2] == 's' && lexeme[3] == 'e') return ELSE;
        if (lexeme[0] == 't' && lexeme[1] == 'r'
            && lexeme[2] == 'u' && lexeme[3] == 'e') return TRUE;
        break;
    case 5:
        if (lexeme[0] == 'w' && lexeme[1] == 'h'
            && lexeme[2] == 'i' && lexeme[3] == 'l'
            && lexeme[4] == 'e') return WHILE;
        if (lexeme[0] == 'p' && lexeme[1] == 'r'
            && lexeme[2] == 'i' && lexeme[3] == 'n'
            && lexeme[4] == 't') return PRINT;
        if (lexeme[0] == 'f' && lexeme[1] == 'a'
            && lexeme[2] == 'l' && lexeme[3] == 's'
            && lexeme[4] == 'e') return FALSE;
        break;
}
```

Como se puede observar, se categorizo las palabras reservadas con respecto a su longitud de cada una, y no es necesario convertir a minúsculas el lexema debido a que MiniC respeta la distinción de mayúsculas y minúsculas, de este modo se pudo ocupar una estructura if para cada caso.

3.3. Estrategia de anticipación para operadores compuestos

Para esta sección, primero se decidió eliminar los caracteres en común de los operadores simples de los operadores compuestos de la función de identificar token simples.

Dicho esto, primero reservamos un lexema con tres espacios en memoria, debido a que lo máximo de caracteres en este tipo de operadores son 2 más el final de cadena.

Primero identificamos si el carácter $c \notin \{=, <, >, !, \&, | \}$, de no ser así, podemos continuar con obtener el siguiente carácter del archivo, primero verificamos si el carácter actual y el carácter posterior forman alguna operación compuesta para regresar el token correspondiente.

En otro caso, lo que se hace es verificar si el carácter actual no es un operador simple y regresar al buffer de lectura el carácter posterior obtenido.

3.4. Distinción entre SLASH y comentarios

Para resolver este requerimiento, fue necesario antes de realizar la lectura de los otros casos, el identificar si el carácter actual es `\`, de ser así, se obtiene el siguiente carácter y se tiene los siguientes casos:

- Si es un `\`, se tienen que leer los siguientes caracteres utilizando nuestra función de avanzar posición de la práctica pasada hasta encontrar los caracteres `\n` o `\r`.
- En otro caso, se regresa el carácter posterior al buffer de lectura del archivo.

3.5. Actualización de línea y columna dentro de comentarios

Como se menciona en la sección anterior, para la lectura de los caracteres que están en una línea de comentario se utilizó nuestra función `static void advance_position(int c, size_t *line, size_t *col)` esto debido a que dentro de esta función se realiza el manejo de la posición de la columna y de la fila en el caso de que se encuentre `\n` o `\r`.

3.6. Generación de tokens ERROR y recuperación

Para esta sección, ya se tenía un buen manejo de los tokens de **ERROR** desde la anterior práctica, debido a que al final de los casos de nuestro lexer se tiene que en dado caso que no este registrado ese carácter, se guarda el lexema y continua con la lectura del archivo.

3.7. Generación y posición de TOKEN EOF

Esta validación se realiza desde la anterior práctica, debido a que al finalizar con el archivo de entrada, se crea un token final de este tipo con la línea y columna final.

3.8. Administración de memoria relevante

Durante el desarrollo del lexer, por cada lexema creado se debía de verificar si al momento de crear espacio reservado en memoria no se producía un error y en este caso detener el programa.

Además de obtener y regresar en ciertas ocasiones el carácter posterior al buffer de lectura de nuestro archivo de entrada.

4. Resultados y pruebas

La implementación se verificó con las pruebas públicas correspondientes de esta práctica las cuales se obtuvieron un 100 % de éxito y para verificar que la funcionalidad con respecto a la práctica anterior continuara, se agregaron las pruebas públicas realizadas durante esa práctica la cual se obtuvo un 100 % de éxito .

Suite	Casos	Resultado
Pruebas públicas	13	13/13 exitosas
Pruebas públicas de la práctica 1	10	10/10 exitosas

[OK] p01_identifiers
 [OK] p02_reserved_words
 [OK] p03_case_sensitive
 [OK] p04_booleans
 [OK] p05_compound_operators
 [OK] p06_incomplete_operators
 [OK] p07_comments
 [OK] p08_comment_at_eof
 [OK] p09_slash_and_comments
 [OK] p10_longest_match
 [OK] p10_mixed_basic
 [OK] p11_error_recovery
 [OK] p12_complete_lexer
 [OK] p13_crlf
 [OK] p1_empty
 [OK] p2_whitespace
 [OK] p3_single_symbols
 [OK] p4_integer
 [OK] p5_integer_expression
 [OK] p6_multiple_lines
 [OK] p7_tabs_spaces
 [OK] p8_error_character
 [OK] p9_negative_number

Aprobadas: 23

Fallidas: 0

5. Problemas encontrados

Las limitaciones conocidas que se mantienen en esta práctica es la compatibilidad con respecto al uso del programa dentro del sistema operativo de Windows y de Linux, lo cual fue resuelto durante la práctica anterior y se documentó su solución propuesta.

6. Conclusiones

La elaboración de esta etapa del compilador como el analizador léxico de MiniC, fue un reto con respecto a la prioridad de los casos al momento de implementarlo en C, donde se tenía que pensar el orden para que nuestro lexer funcionará con respecto a las reglas dichas durante la práctica.

De este modo, al revisar los resultados obtenidos podemos estar satisfechos a que nuestra propuesta de lexer pueda apoyar en las siguientes etapas de evaluación de nuestro compilador al facilitar la lectura del archivo y empezar a realizar el análisis sintáctico del programa.

7. Uso de herramientas de Inteligencia Artificial

No se utilizó ninguna herramienta de inteligencia artificial para la elaboración de la práctica.

8. Referencias

- Pablo Martínez, Y. J.; Salazar González, P. Y. (2026). Compiladores - Práctica 2, semestre 2027-1. Facultad de Ciencias, UNAM.